

Figure 2: Our first 'Hello World' program

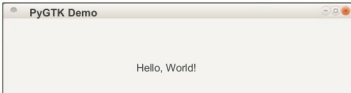


Figure 3: The 'Hello World' program after modification

```
import gtk
wnd = gtk.Window()
lbl = gtk.Label("Hello, World!")
wnd.add(lbl)
wnd.show_all()
wnd.connect("destroy", gtk.main_quit)
gtk.main()
```

The first line, `import gtk`, tells Python to use the PyGTK library. (Note that we are using GTK+ 2 here. GTK+ 3 is available, but we won't use it here since it may not be present in older systems. I recommend that you migrate to GTK+ 3 after getting familiar with what we do in this tutorial.)

We've created two objects, `wnd` and `lbl`, instances of the GTK window and GTK label classes. To put it in simple terms, we created a GTK window and a GTK label, named `wnd` and `lbl`, respectively.

The window is the top-level widget that we present to the user. We've to place all other controls on it. That's what we have done with the line `wnd.add(lbl)`.

By default, all widgets are hidden. If we give the command `wnd.show()`, only the window will become visible. Then we have to give the command `lbl.show()`. If there are a dozen controls, we have to do this for each of them. Thanks to the `show_all()` function, all these tasks are simplified with a single command, `wnd.show_all()`. Now, `wnd` and everything it contains is made visible.

`gtk.main()` starts the main loop of GTK+, which we mentioned earlier while discussing the workflow of a GUI program. If you remove this line, the program won't work. This loop runs the GUI program and keeps it alive forever. To break that loop when the user presses the window's *Close* button, we have to make a connection. That's what the line `wnd.connect("destroy", gtk.main_quit)` is for.

Adjusting the properties

Now let's modify our 'Hello World' program a bit. Let's

change the title and size of the window in order to learn how to change the properties of widgets.

Add these two lines following the line `lbl =` in our previous program:

```
wnd.set_title("PyGTK Demo")
wnd.resize(400, 300)
```

Run the program and you will see that our window has got a title and perfect sizing. You can use the function `set_size_request` instead of `resize`. The former sets a minimum size for the window, so the user can't compress it any more.

Placing multiple widgets

In the previous example, we placed only a single widget on the window (GTK label). But we have to deal with multiple widgets in a practical program. So how does one do this? Let's create two widgets, write commands to add them both, and check whether it works.

```
import gtk
wnd = gtk.Window()
lbl1 = gtk.Label("This is label 1")
lbl2 = gtk.Label("This is label 2")
wnd.add(lbl1)
wnd.add(lbl2)
wnd.show_all()
wnd.connect("destroy", gtk.main_quit)
gtk.main()
```

But what we now get is a warning.

GtkWarning: Attempting to add a widget with type *GtkLabel* to a *GtkWindow*, but as a *GtkBin* subclass a *GtkWindow* can only contain one widget at a time; it already contains a widget of type *GtkLabel*—`wnd.add(lbl2)`

It clearly states that `GtkWindow` cannot have multiple controls at a time. Then how do we create a practical program? The solution is to use a *container*. A container is a widget that can have multiple widgets as its children, including other containers. GTK+ presents a variety of containers, which include `Box`, `Grid`, `Menu Bar`, `Layout` and `Scrolled Window`. `VBox` and `HBox` help us to arrange widgets in vertical and horizontal directions, respectively. `Grid` makes a tabular arrangement possible (i.e., rows and columns). `Menu Bar` needs no introduction. `Scrolled Window` can be used to contain a growing widget. For example, the main text area of a text editor is put inside a `Scrolled Window`. Note that `GtkScrolledWindow` is not a type of `GtkWindow`, but a simple widget which can be put inside any container, including `GtkWindow`. There are some other kinds of containers that are not mentioned in this article.